Table of contents

Introduction..... 3
i) C Source Code.....3
ii) Terminal Execution & Verification..... 3
Task 2: Process Identity (PID AND PPID)..... 4
i) C Source Code.....4
ii) Terminal Execution & Verification..... 5
Task 3: Exit Codes and OS Feedback.....5
i) C Source Code.....5
ii) Terminal Execution & Verification..... 6
Task 4: Standard I/O Streams.....6
i) C Source Code.....6
ii) Terminal Execution..... 7
Task 5: Conditional Execution and Termination..... 7
i) C Source Code.....7
ii) Terminal Execution & Verification..... 8
Knowledge Test:..... 8
Conclusion.....10

List of Figures

Figure 1: task1_alive.c file code..... 3
Figure 2: Terminal 1: Run in background..... 4
Figure 3: Terminal 2: Monitor the process..... 4
Figure 4: task2_identity.c..... 4
Figure 5: Terminal commands..... 5
Figure 6: task3_exit.c..... 5
Figure 7: Checking Exit Status..... 6
Figure 8: task4_input.c..... 6
Figure 9: Running Task 4..... 7
Figure 10: task5_control.c..... 7
Figure 11: Testing Control Flow..... 8

Introduction

In Lecture 3, we learned that an OS manages processes and resources. In this lab, we bring that theory to life. Instead of just writing code that prints text, we will write code that interacts with the Linux kernel to query its own process identity, control its execution time, and report its success or failure back to the OS shell.

Task 1: The Long-Running Process

OS Insight: In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

i) C Source Code

We create a program that runs for 30 seconds using a loop and “sleep()”.

```
  GNU nano 8.7.1                task1_alive.c *
#include <stdio.h>
#include <unistd.h>

int main() {
        printf("I am starting...\n");

        // Loop for 30 seconds
        for(int i = 1; i <= 30; i++) {
                sleep(1); // Pauses execution for 1 second
        }

        printf("I am finished.\n");
        return 0;
}
```

Figure 1: task1_alive.c file code.

ii) Terminal Execution & Verification

To make this interesting, we run the program in the back-ground using & so we can use a second terminal to inspect it while it’s alive.

```
root@DESKTOP-OAP8OK0:/batch38# gcc task1_alive.c -o task1
root@DESKTOP-OAP8OK0:/batch38# ./task1 &
[1] 3197
I am starting...
root@DESKTOP-OAP8OK0:/batch38# I am finished.

[1]+  Done                    ./task1
```

Figure 2: Terminal 1: Run in background.

```
root@DESKTOP-OAP8OK0:/# ps aux | grep task1
root        3222  0.0  0.0   4128  2464 pts/2    S+   15:30   0:
00 grep --color=auto task1
```

Figure 3: Terminal 2: Monitor the process.

Task 2: Process Identity (PID AND PPID)

OS Insight: Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually your terminal shell.

i) C Source Code

```
  GNU nano 8.7.1              task2_identity.c *
#include <stdio.h>
#include <unistd.h>

int main() {
        // Get current Process ID
        pid_t my_pid = getpid();

        // Get Parent Process ID
        pid_t my_ppid = getppid();

        printf("My PID is: %d\n", my_pid);
        printf("My Parent PID is: %d\n", my_ppid);

        printf("Sleeping for 20 seconds...\n");
        sleep(20);

        return 0;
}
```

Figure 4: task2_identity.c

ii) Terminal Execution & Verification

```
root@DESKTOP-OAP8OK0:/batch38# gcc task2_identity.c -o task2
root@DESKTOP-OAP8OK0:/batch38# ./task2 &
[1] 3308
My PID is: 3308
My Parent PID is: 655
Sleeping for 20 seconds...
root@DESKTOP-OAP8OK0:/batch38# ps -p 3308 -o pid,ppid,cmd
    PID    PPID CMD
[1]+  Done                    ./task2
```

Figure 5: Terminal commands

Task 3: Exit Codes and OS Feedback

OS Insight: When a program finishes, it returns an integer to the OS. 0 means "Success", and any non-zero number (like 1) means "Error/Failure". The shell stores this in $?.

i) C Source Code

```
  GNU nano 8.7.1                                          task3_exit.c *
#include <stdio.h>

int main() {
        int num;
        printf("Enter a number (positive for success, negative for fail): ");
        scanf("%d", &num);
        if (num > 0) {
           printf("Success\n");
           return 0; // Tell OS: Success
        } else {
           printf("Failure\n");
           return 1; // Tell OS: Error
        }
}
```

Figure 6: task3_exit.c

ii) Terminal Execution & Verification

```
root@DESKTOP-OAP8OK0:/batch38# gcc task3_exit.c -o task3
root@DESKTOP-OAP8OK0:/batch38# ./task3
Enter a number (positive for success, negative for fail): 1
Success
root@DESKTOP-OAP8OK0:/batch38# ./task3
Enter a number (positive for success, negative for fail): 5
Success
root@DESKTOP-OAP8OK0:/batch38# echo $?
0
root@DESKTOP-OAP8OK0:/batch38# ./task3
Enter a number (positive for success, negative for fail): -5
Failure
root@DESKTOP-OAP8OK0:/batch38# echo $?
1
```

Figure 7: Checking Exit Status

Task 4: Standard I/O Streams

OS Insight: The OS provides standard input (stdin) and standard output (stdout) streams. “scanf” reads from stdin, and “printf” writes to stdout.

i) C Source Code

```
  GNU nano 8.7.1                task4_input.c *
#include <stdio.h>

int main() {
        char name[50];

        printf("Enter your name: ");
        // Read string from standard input
        scanf("%s", name);

        printf("Hello, %s! Welcome to OS Class.\n", name);
        return 0;
}
```

Figure 8: task4_input.c

ii) Terminal Execution

```
root@DESKTOP-OAP8OK0:/batch38# gcc task4_input.c -o task4
root@DESKTOP-OAP8OK0:/batch38# ./task4
Enter your name: Cheten
Hello, Cheten! Welcome to OS Class.
```

Figure 9: Running Task 4

Task 5: Conditional Execution and Termination

OS Insight: Programs often branch based on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.

i) C Source Code

```
 GNU nano 8.7.1                                          task5_control.c *
#include <stdio.h>
#include <unistd.h>

int main() {
        int choice;

        // Print PID at start
        printf("Current PID: %d\n", getpid());

        printf("Do you want to continue? (1 for Yes, 0 for No): ");
        scanf("%d", &choice);

        if (choice == 1) {
                printf("Continuing...\n");
                sleep(5);
                return 0; // Success
        } else {
                printf("Exiting...\n");
                return 1; // Failure/Abort
        }
}
```

Figure 10: task5_control.c

ii) Terminal Execution & Verification

```
root@DESKTOP-OAP8OK0:/batch38# gcc task5_control.c -o task5
root@DESKTOP-OAP8OK0:/batch38# ./task5
Current PID: 3705
Do you want to continue? (1 for Yes, 0 for No): 1
Continuing...
root@DESKTOP-OAP8OK0:/batch38# echo $?
0
root@DESKTOP-OAP8OK0:/batch38# ./task5
Current PID: 3724
Do you want to continue? (1 for Yes, 0 for No): 0
Exiting...
root@DESKTOP-OAP8OK0:/batch38# echo $?
1
```

Figure 11: Testing Control Flow

Knowledge Test:

1. What is the difference between source code and an executable?
Ans: The difference between source code and executable code is:
Source code: Human readable program (like C language)
Executable: Machine code file created by compiler, stored on disk, ready to run.

---

2. What command compiles a C program?
Ans: gcc task1_alive.c -o task1

---

3. What is a process?
Ans: A process is a program in execution, loaded into RAM, managed by the OS.

---

4. What does PID stand for?
Ans: Process ID. A unique number assigned by the OS to track each process.

---

5. How do you check the exit code of the last command?
Ans: echo $? in the shell.

---

6. What does return 0; mean in main()?
Ans: Return 0 in main() means it returns success. It signals normal exit(success) to the OS.

---

7. What system call creates a new process?
Ans: fork() system call.

---

8. What system call replaces the current process with a new program?
Ans: execve() system call.

---

9. How do you see running processes in the terminal?
Ans: By using commands like ps or top.

---

10. What happens to memory when a process ends?
Ans: OS frees RAM, closes files, removes PID from table.

---

11. Where is your program stored before execution?
Ans: On disk as an executable file.

---

12. Where is your program stored during execution?
Ans: In RAM (main memory) as a process.

---

13. Why does the OS assign a PID to each process?
Ans: To track, schedule, and manage processes uniquely.

---

14. When does a process end?
Ans: When it returns/exit(0), receives a signal (Ctrl+C), or crashes (segfault, divide by zero).

---

15. Who assigns the PID?
Ans: The Operating System kernel assigns the PID when creating the process.

Conclusion

This lab demonstrated that C programs do not exist in a vacuum; they are active processes managed by the Linux kernel. By utilizing system calls like “getpid()”, “sleep()”, and return codes, we successfully bridged the gap between high-level C code and low-level OS process management.